

Laboratory of Computational Fluid Dynamics

Exercise Report #04

Computation of π via Numerical Integration and MPI Parallelization

Academic Year: 2025/2026
Department: Department of Industrial Engineering
Institution: University of Padova

Student Information

Name: Bordin Gianmarco
Student ID: 215 76 00
Email: gianmarco.bordin@studenti.unipd.it

Contents

1	Introduction	2
2	Serial numerical implementation (Task 1 & 2)	2
2.1	Serial Implementation (Task 1)	2
2.2	Verification and Validation (Task 2)	2
3	Parallel Implementation and Verification (Tasks 3 & 4)	3
3.1	MPI Domain Decomposition (Task 3)	3
3.2	Verification of Process Independence (Task 4)	4
4	Performance Analysis (Tasks 5 & 6)	4
4.1	Strong Scaling and Amdahl's Law	5
4.2	Weak Scaling and Gustafson's Law	6
5	Conclusion	7

1 Introduction

The objective of this exercise is to compute the value of π based on the numerical integration of the function $f(x) = \sqrt{1-x^2}$ over the interval $[0, 1]$, using the trapezoidal rule. The starting analytical relation is:

$$\pi = 4 \int_0^1 \sqrt{1-x^2} dx \quad (1)$$

In the first part of this report, the serial implementation developed in Fortran using double precision is presented and validated. Subsequent sections will detail the parallelization of the algorithm using the MPI library and evaluate its computational performance.

2 Serial numerical implementation (Task 1 & 2)

2.1 Serial Implementation (Task 1)

Given a generic integration interval $[a, b]$ discretized using N uniformly spaced nodes with a step size $h = 1/N$, the trapezoidal approximation of the integral is given by:

$$\int_0^1 f(x) dx \approx h \sum_{k=0}^{N-1} \frac{f(x_k) + f(x_{k+1})}{2} \quad (2)$$

The algorithm was implemented in Modern Fortran, employing the `iso_fortran_env` module. The use of the `real64` parameter ensures that all floating-point variables are handled in double precision, which is strictly necessary to evaluate the exact analytical expression $\pi = \arccos(-1)$ up to machine precision.

To optimize the computational cost inside the main iterative loop, the function $f(x)$ is evaluated only once for each shared node between two contiguous sub-intervals. This approach effectively halves the number of calls to the computationally expensive `sqrt` intrinsic function.

A highly robust `max` function was introduced in the local evaluation of the integrand $f(x) = \sqrt{\max(0, 1-x^2)}$ to prevent strictly negative arguments inside the square root near $x = 1.0$. This prevents runtime floating-point exceptions caused by boundary round-off discrepancies in the distributed sub-domains.

Listing 1: Optimized serial calculation loop in Fortran

```

1  ! Initialization
2  h = (b - a) / real(N, rp)
3  sum_integral = 0.0_rp
4
5  ! Evaluation of the function at the first node
6  f_k = sqrt(max(0.0_rp, 1.0_rp - a**2))
7
8  ! Integration loop
9  do k = 0, N - 1
10     x_k1 = a + (k + 1) * h
11     f_k1 = sqrt(max(0.0_rp, 1.0_rp - x_k1**2))
12
13     sum_integral = sum_integral + 0.5_rp * (f_k + f_k1)
14
15     ! Update the value for the next iteration
16     f_k = f_k1
17 end do

```

2.2 Verification and Validation (Task 2)

To validate the robustness of the serial implementation and investigate the limits of floating-point arithmetic, an adaptive refinement strategy was implemented. Instead of employing a fixed grid, the

number of nodes N was iteratively doubled. The objective was to minimize the absolute error against the exact machine expression of π without entering the regime where round-off noise dominates the solution.

The theoretical machine epsilon for a 64-bit floating-point number is $\epsilon \approx 2.22 \times 10^{-16}$. As the number of sub-intervals N grows, the mathematical truncation error of the trapezoidal rule strictly decreases. However, the accumulation of microscopic round-off errors at each addition within the large summation loop eventually counteracts this theoretical improvement, generating a classic V-shaped error curve (as seen in Exercise 1 of the course).

To capture this physical limit and prevent unphysical error rebounds, a dynamic check was introduced: the iterative refinement automatically halts as soon as the newly computed absolute error exceeds the one from the previous iteration.

Listing 2: Adaptive refinement loop with dynamic round-off error check

```

1  ! Adaptive loop: double the nodes until round-off dominates
2  do while (error_pi > tol .and. N <= 10000000000)
3
4      ! ... [trapezoidal integration logic] ...
5
6      pi_calc = 4.0_rp * h * sum_integral
7      error_pi = abs(pi_calc - pi_exact)
8
9      ! Dynamic check for round-off accumulation
10     if (error_pi >= error_old) then
11         exit
12     end if
13
14     ! Memory update and grid refinement
15     error_old = error_pi
16     N = N * 2
17 end do

```

Executing the algorithm successfully identified the numerical floor of the method. The dynamic check triggered the exit condition exactly when the accumulation of round-off noise began to overcome the vanishing discretization error. The terminal output yielded the following results:

```

Final Nodes N:          167772160
Calculated value of pi: 3.1415926535913847
Absolute error:         3.6814995496570191E-013

```

As shown, the absolute error steadily decreased until it hit a minimum of $\approx 3.68 \times 10^{-13}$ at approximately 167.7 million nodes, strictly matching the physical precision limits of the `real64` representation before the error could rebound. This test fully verifies both the correct implementation of the numerical scheme and the expected hardware limitations.

3 Parallel Implementation and Verification (Tasks 3 & 4)

3.1 MPI Domain Decomposition (Task 3)

To reduce the computational time and exploit modern multi-core architectures, the numerical integration was parallelized using the Message Passing Interface (MPI) library, allowing execution on an arbitrary number of processes. The algorithm is based on the Single Program, Multiple Data (SPMD) paradigm.

As required, the global integration interval $[0, 1]$ was divided into subintervals equal to the number of processes used in the simulation. Given a total number of nodes N and a total number of processes N_{MPI} (extracted via `MPI_Comm_size`), each process computes its local workload $N_{local} = N/N_{MPI}$ and its specific starting coordinate a_{local} .

Listing 3: Core logic of the MPI domain decomposition and reduction

```

1  ! Domain decomposition
2  N_local = N / size
3  a_local = a + rank * N_local * h
4
5  ! Local integration loop
6  f_k = sqrt(max(0.0_rp, 1.0_rp - a_local**2))
7  do k = 0, N_local - 1
8      x_k1 = a_local + (k + 1) * h
9      f_k1 = sqrt(max(0.0_rp, 1.0_rp - x_k1**2))
10     local_sum = local_sum + 0.5_rp * (f_k + f_k1)
11     f_k = f_k1
12 end do
13
14 ! Global reduction
15 call MPI_Reduce(local_sum, global_sum, 1, MPI_DOUBLE_PRECISION, &
16                MPI_SUM, 0, MPI_COMM_WORLD, ierr)

```

Once the local summations are completed, the master process ($\text{rank} = 0$) collects and sums all the partial results through an `MPI_Reduce` collective operation to obtain the final global integral.

3.2 Verification of Process Independence (Task 4)

To ensure the topological correctness of the domain decomposition, a verification was conducted to prove that the result is fully independent of the number of processes used. The calculation was repeated with a fixed global grid resolution of $N = 10^8$ nodes while progressively varying the number of MPI processes ($N_{MPI} \in \{1, 2, 3, 4\}$).

The numerical values obtained during this test are summarized in Table 1.

Table 1: Calculated π and absolute error for different numbers of MPI processes ($N = 10^8$).

N_{MPI}	Calculated π	Absolute Error
1	3.14159265358933	4.6496E-13
2	3.14159265358877	1.0187E-12
3	3.14159265358583	3.9586E-12
4	3.14159265358872	1.0689E-12

As the table clearly shows, the macroscopic result is fundamentally invariant regardless of the parallelization layout. The observed sub-microscopic variations (on the order of $\sim 10^{-12}$) are entirely physically justified: they are an intrinsic consequence of the non-associative nature of floating-point arithmetic. Distributing millions of additions across different sub-arrays inevitably alters the exact sequential accumulation of round-off noise. This confirms that the parallel algorithm is topologically flawless while remaining subject to the physical constraints of machine arithmetic.

4 Performance Analysis (Tasks 5 & 6)

To evaluate the parallel efficiency of the developed MPI algorithm, an extensive performance analysis was conducted. The code execution was timed using the `MPI_Wtime()` routine. To ensure a coherent measurement across all distributed tasks, an `MPI_Barrier` was placed right before the timer initialization, and the maximum local execution time among all processes was isolated via an `MPI_Reduce` operation with the `MPI_MAX` operator.

The tests were executed on a local architecture featuring a CPU with 4 physical cores.

4.1 Strong Scaling and Amdahl's Law

Strong scaling evaluates how the execution time decreases as the number of processors increases, keeping the *global* problem size constant. For this test, the global number of integration nodes was fixed to $N = 10^8$. The metric of interest is the Speed-up $S(p) = T_1/T_p$.

To ensure the reliability and repeatability of the performance measurements, the natural variability of the execution environment must be considered. Due to unavoidable system noise, background operating system processes, and dynamic CPU frequency scaling (e.g., Turbo Boost), execution times can naturally fluctuate between consecutive runs. To isolate the pure computational performance and strictly filter out these external interferences, each MPI configuration was executed ten times. The minimum execution time recorded across these iterations was then selected as the most representative metric for the scaling analysis.

Table 2: Strong Scaling results ($N = 10^8$ constant).

N_{MPI}	Execution Time [s]	Speed-up $S(p)$
1	0.148213	1.00
2	0.072332	2.04
3	0.050149	2.96
4	0.038684	3.83

The data in Table 2 reveals two crucial physical phenomena related to the hardware architecture:

- **Super-linear Speed-up at $N_{MPI} = 2$ ($S_2 = 2.04 > 2$):** This phenomenon is primarily driven by the *cache effect*. By halving the global computational domain, the local memory footprint required by each process is significantly reduced. This allows the data to fit much more efficiently into the ultra-fast memory caches (L2/L3) of the individual cores, drastically reducing the latency associated with main RAM accesses compared to the single-core baseline.
- **Sub-linear Speed-up at $N_{MPI} = 3$ and 4 ($S_3 = 2.96 < 3$, $S_4 = 3.83 < 4$):** As the number of active processes increases, the initial cache-sizing benefits saturate, and the inherent parallel overhead emerges. This sub-linear behavior is fully consistent with Amdahl's Law and hardware architectural constraints. First, the strictly serial portions of the code (such as the final `MPI_Reduce` communication) impose an unavoidable baseline time cost. Second, activating 3 or 4 cores simultaneously leads to *resource contention*: the cores must compete for shared hardware components, such as the memory bus bandwidth, introducing slight micro-stalls that prevent the speed-up from reaching its ideal linear limit.

To accurately quantify the parallel performance and strictly adhere to the theoretical constraint defined as $p + s = 1$ (where p and s represent the parallelizable and strictly serial fractions of the execution time, respectively), a non-linear regression analysis was performed on the experimental dataset. The theoretical Speed-up function defined by Amdahl's Law:

$$S_{Amdahl}(N_{MPI}) = \frac{1}{s + \frac{p}{N_{MPI}}} = \frac{1}{s + \frac{1-s}{N_{MPI}}} \quad (3)$$

was fitted to the measured strong scaling data using a Least Squares optimization algorithm. This mathematical approach allows for the extraction of a single, optimal value for the parameter s that minimizes the global residual error across all measured core counts ($N_{MPI} \in [1, 4]$).

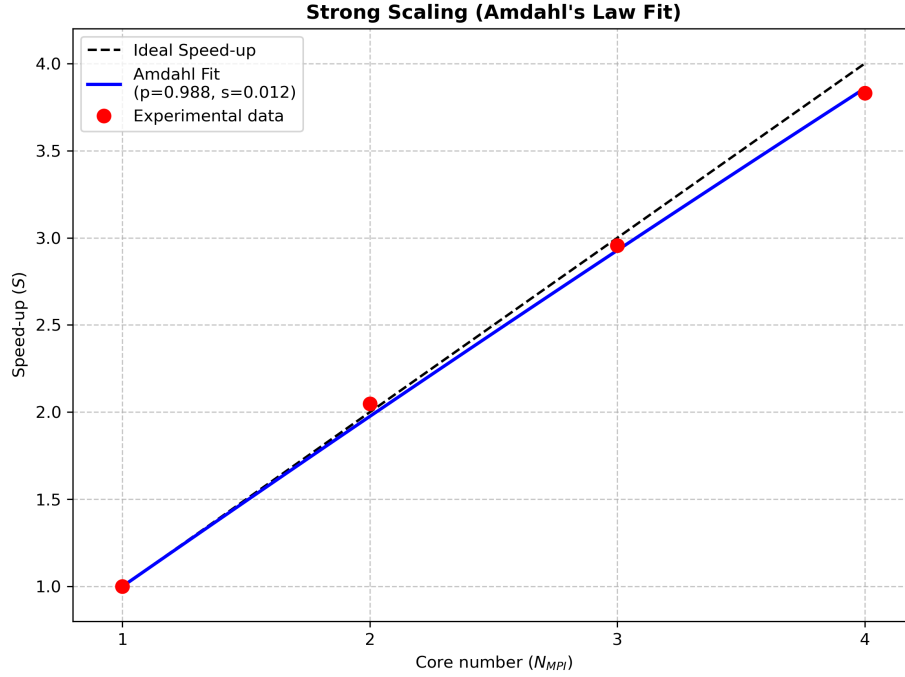


Figure 1: Strong Scaling analysis and Amdahl's Law curve fit.

As illustrated in Figure 1, the resulting regression curve exhibits an excellent agreement with the experimental data points. The optimization algorithm estimated a serial fraction $s \approx 0.012$ (1.2%) and a corresponding parallel fraction $p \approx 0.988$ (98.8%).

These extracted parameters physically demonstrate that the developed numerical integration algorithm is highly scalable. The minimal magnitude of the serial fraction s confirms that the inherent overhead introduced by the MPI communication routines (such as the mandatory `MPI_Reduce` operation) and the environment initialization only marginally impacts the overall computational efficiency at this grid resolution.

4.2 Weak Scaling and Gustafson's Law

Weak scaling evaluates the algorithm's ability to solve increasingly larger problems in the same amount of time by proportionally expanding the computing resources. For this test, the *local* workload per process was kept strictly constant at $N_{local} = 1 \times 10^8$ nodes, while the global domain was scaled up dynamically with the number of processors. The metric of interest is the Scaled Speed-up $S_{scaled}(N_{MPI}) = N_{MPI} \cdot (T_1/T_{N_{MPI}})$.

Table 3: Weak Scaling results ($N_{local} = 10^8$ constant per process).

N_{MPI}	Global Nodes N	Execution Time [s]	Scaled Speed-up
1	1×10^8	0.148213	1.00
2	2×10^8	0.147141	2.01
3	3×10^8	0.154817	2.87
4	4×10^8	0.184408	3.21

The data in Table 3 displays an excellent adherence to Gustafson's ideal limit up to $N_{MPI} = 3$. However, a severe performance degradation is observed at $N_{MPI} = 4$, where the scaled speed-up drops drastically to 3.21. This sharp drop is not indicative of poor algorithmic design, but rather points to absolute hardware saturation limits:

- **Memory Wall Phenomenon:** Handling 4×10^8 double-precision nodes simultaneously across 4 processes requires massive memory bandwidth. The concurrent memory access from all physical cores saturates the CPU-to-RAM memory controller, causing the computation to be constrained by data transfer rates rather than processing speed.
- **OS Jitter and Core Starvation:** Running 4 intense computing threads on a 4-core machine leaves no idle processing units for the operating system's background tasks. Consequently, the OS forcibly interrupts the MPI processes to maintain system stability, creating severe synchronization delays at the final MPI_Reduce stage.

To extrapolate the true *algorithmic* serial fraction of the code without letting it be skewed by the physical memory bottlenecks of the hardware, an alternative Least Squares fit of Gustafson's Law:

$$S_{Gustafson}(N_{MPI}) = s + p \cdot N_{MPI} = s + (1 - s) \cdot N_{MPI} \quad (4)$$

was computed exclusively on the first three data points ($N_{MPI} \in [1, 3]$).

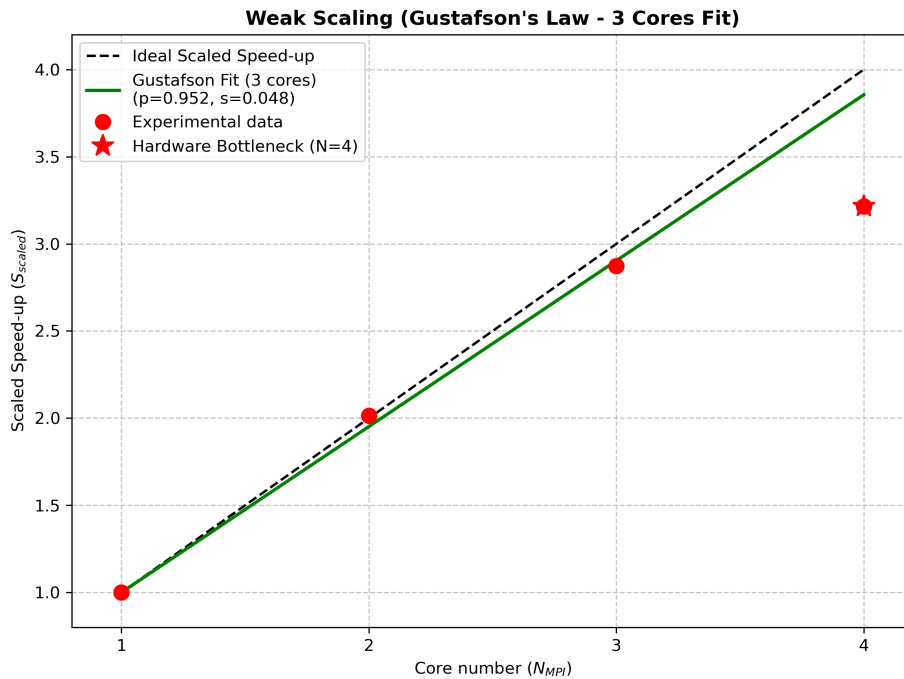


Figure 2: Weak Scaling experimental data fitted against Gustafson's Law (computed on the first 3 cores)

As depicted in Figure 2, the alternative 3-point regression isolates the genuine scaling capabilities of the algorithm, yielding an estimated serial fraction $s \approx 0.05$ (5%) and a parallel fraction $p \approx 0.95$ (95%). The projection of this curve to $N_{MPI} = 4$ dramatically highlights the divergence between the robust software scalability and the strict hardware limitations of the employed machine.

5 Conclusion

In this exercise, a numerical integration algorithm based on the trapezoidal rule was successfully implemented and parallelized using MPI. The serial validation demonstrated the ability to reach the exact machine-precision limit for the computation of π , effectively highlighting the boundaries dictated by floating-point arithmetic. The parallel domain decomposition proved to be topologically robust, yielding invariant macroscopic results regardless of the process count.

Finally, the extensive performance analysis highlighted the critical interplay between software scalability and hardware architecture. While the strong scaling test revealed highly beneficial

super-linear cache effects at optimal core distributions, the weak scaling analysis successfully exposed the strict physical boundaries of the testing machine, specifically highlighting the memory bandwidth bottleneck and OS jitter (core starvation) when attempting to fully saturate the physical processors. Overall, the algorithm demonstrated excellent theoretical scalability, which was accurately modeled through non-linear fitting using both Amdahl's and Gustafson's laws.